

PRIMERA SUSTENTACIÓN OFICIAL

Una ruta reproducible de Windows a Android ARM64

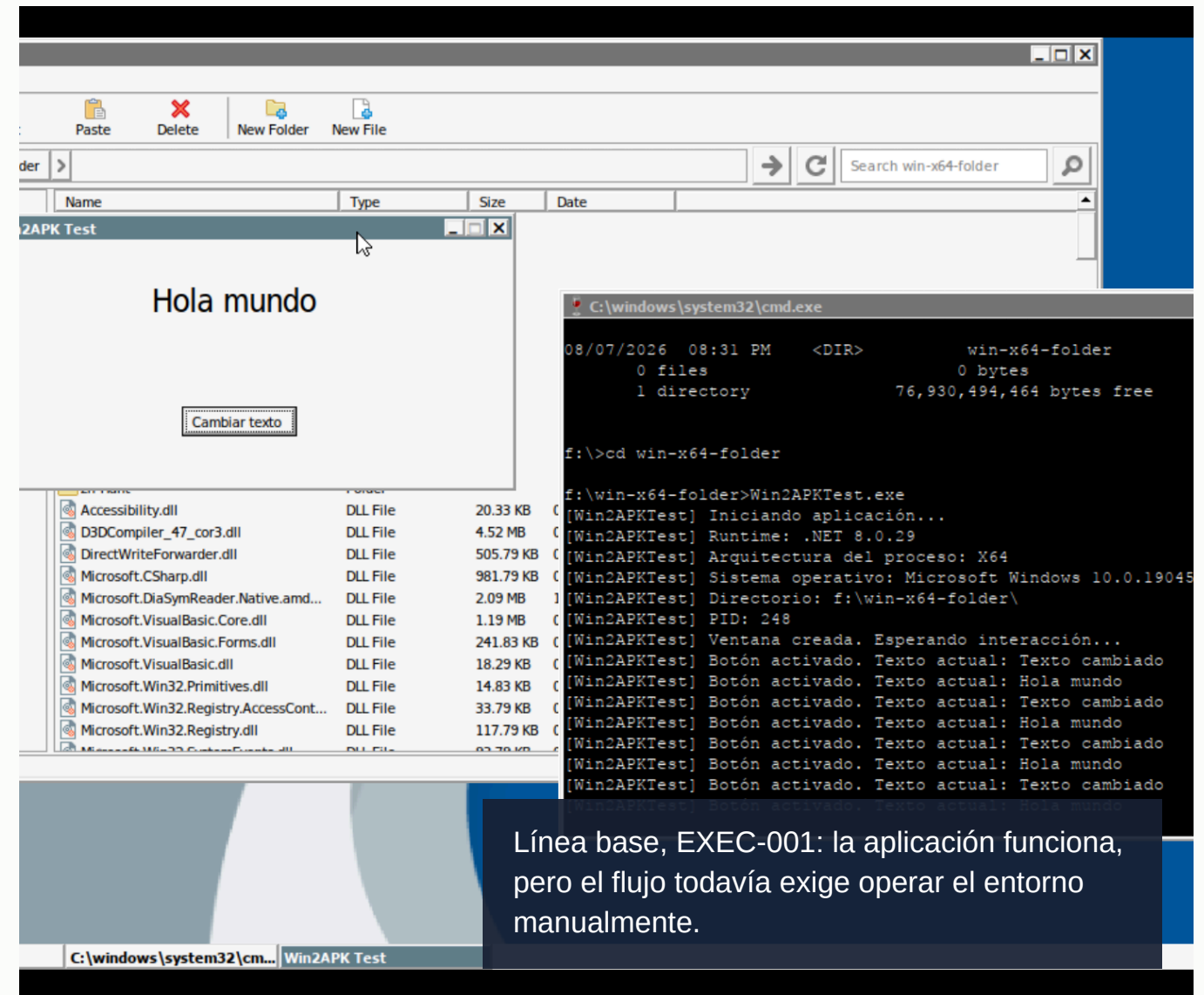
Empaquetado, distribución y ejecución mediante un entorno de compatibilidad.



Ejecutar no basta si el usuario debe reconstruir el entorno

¿Cómo pasar de una carpeta Windows a una experiencia Android **instalable y repetible**?

- 01 Arquitectura x86-64 frente a ARM64
- 02 Dependencias, gráficos y configuración por aplicación
- 03 Distribución de payloads que pueden ocupar gigabytes



Win2APK convierte decisiones técnicas en una canalización verificable

ENTRADA

Aplicación Windows preparada

Ejecutable, DLL, recursos, configuración y condiciones conocidas.



SALIDA

AAB + APK set + reporte

Artefactos firmados, configuración embebida, inventario y hashes.

Objetivo actual: automatizar el empaquetado y la preparación del entorno para distribuir y ejecutar aplicaciones Windows compatibles en Android ARM64 con menos configuración manual.

Winlator aporta el entorno; Win2APK industrializa el recorrido

Winlator / componentes base

Capacidad de compatibilidad y ejecución

- Root filesystem Linux y contenedores
- Wine, Box64 y ruta gráfica
- Controles y servicios de ejecución

Trabajo de Win2APK

Automatización, encapsulamiento y evidencia

- CLI en Rust: validar, planear, construir e instalar
- Identidad, configuración y arranque directo por aplicación
- Partición del payload, firma, limpieza y trazabilidad
- Perfiles gráficos, interfaz adaptativa y medición ADB

Dos momentos: construir el paquete y preparar la ejecución

En Linux x86-64 · Build time

Win2APK CLI

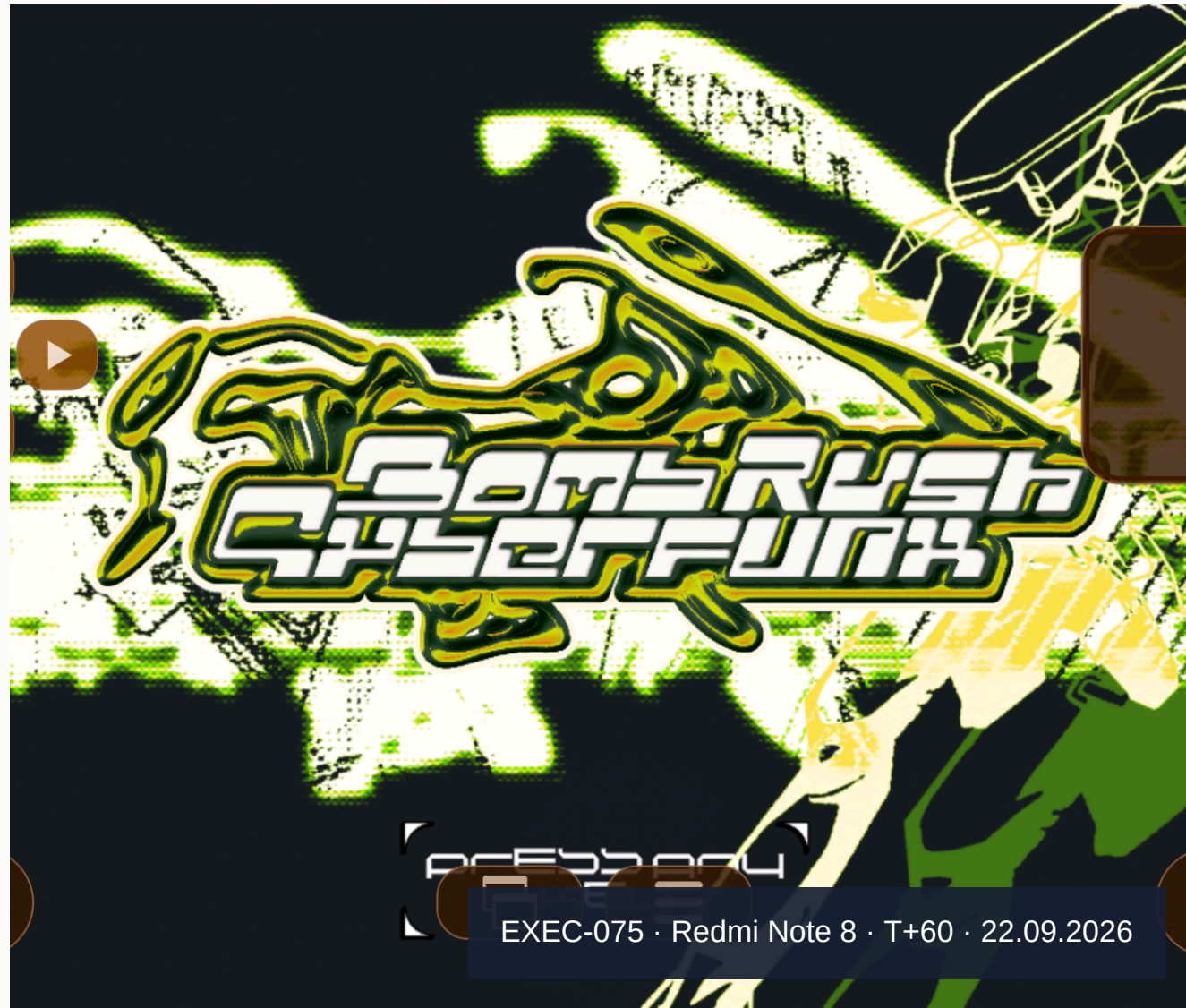
- 01 Validar e inventariar**
JSON v2, archivos, tamaño y SHA-256
- 02 Planear y segmentar**
Módulo base y asset packs
- 03 Configurar y firmar**
Identidad, runtime, icono y claves
- 04 Producir artefactos**
AAB, APKs, reporte y manifiesto

En Android ARM64 · Run time

Winlator Core configurado

- 05 Entregar el payload**
Play Asset Delivery o pruebas locales
- 06 Preparar contenedor**
Rootfs, ruta Windows y acceso directo
- 07 Traducir y compatibilizar**
Box64: x86-64 → ARM64 · Wine: Win32 → POSIX
- 08 Renderizar**
DXVK / WineD3D → Vulkan / OpenGL → GPU

El paquete abre la aplicación objetivo y conserva controles del runtime



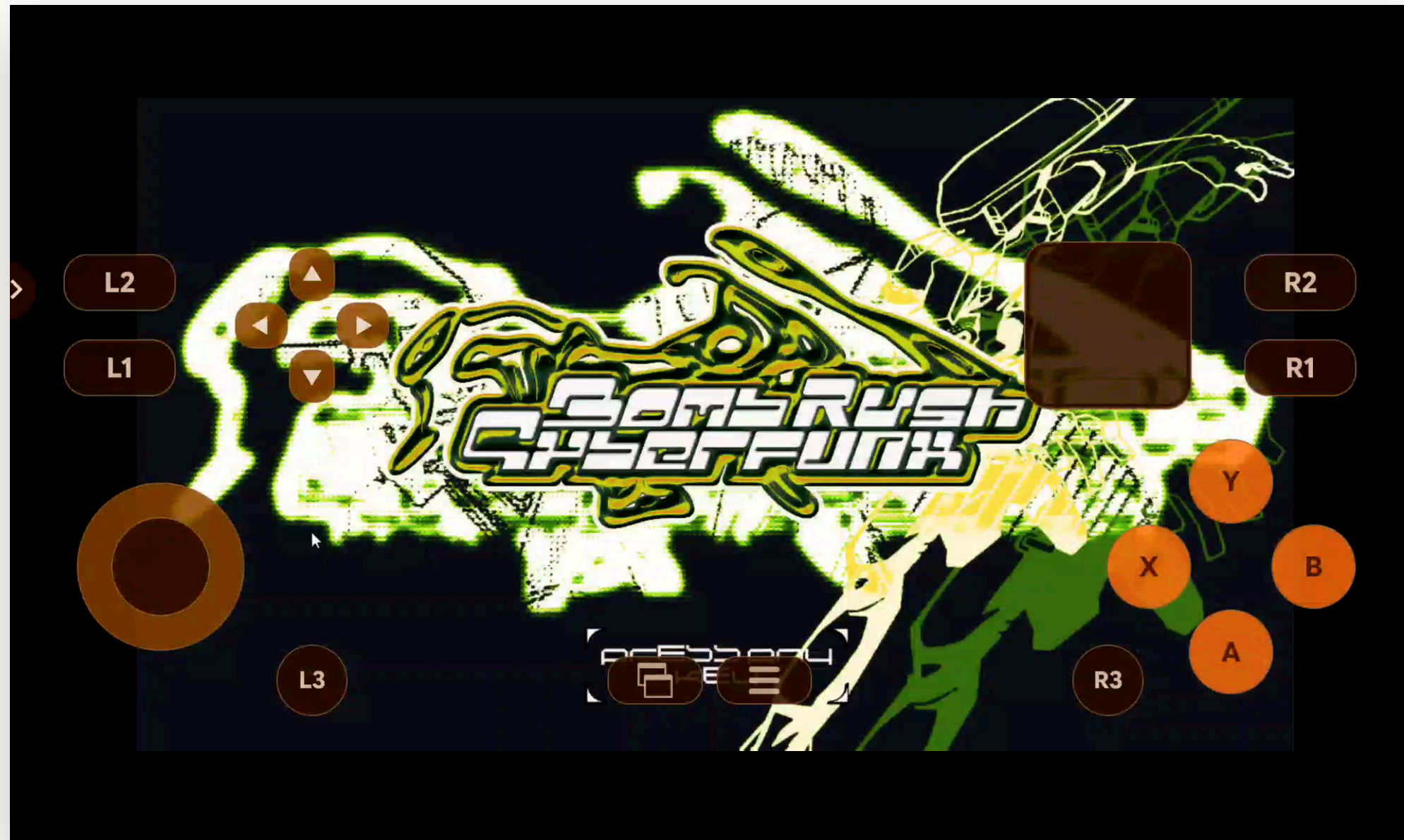
2.275 s

Proceso del juego visible después del lanzamiento del paquete preinstalado.

120 s

Aplicación y juego observables durante toda la ventana; pantalla de título y gamepad visibles.

De un icono Android a una aplicación Windows con controles táctiles



RUTA DE CONTINGENCIA

- 01 Abrir el paquete en Redmi Note 8.
- 02 Confirmar título y gamepad.
- 03 Si falla: tecla V y continuar con este video.

Cada afirmación se ata a una ejecución, un dispositivo y una evidencia

01	02	03	04	05
Definir escenario y estado esperado	Fijar artefacto, hash y condiciones	Ejecutar y medir con ADB	Clasificar captura T+60 y logs	Separar observación, interpretación y decisión

DISPOSITIVO	GPU / FAMILIA	EVIDENCIA DISPONIBLE
Pixel 9a	Tensor / Mali	Perfiles gráficos y memoria
Redmi Note 8	Snapdragon / Adreno	Arranque, UI y gamepad
Xiaomi Mi A3	Snapdragon / Adreno	PAD y payload grande
Lenovo TB-J606F	Snapdragon / Adreno	Tablet, Core y fallos

815

archivos y 5.847 GB entregados en la validación del payload de Cuphead en EXEC-049.

CoreCLR falló primero; el diagnóstico cambió el empaquetado

INTENTAMOS	OBSERVAMOS	INVESTIGAMOS	CAMBIAMOS	RESULTADO
Ejecutar la publicación .NET inicial dentro de Winlator.	Fallo de inicialización del heap y salida de CoreCLR.	Memoria del GC, modo de publicación y configuración del runtime.	Publicación self-contained, GC workstation no concurrente y límite de heap.	Win2APKTest abrió y respondió a interacción en Lenovo.

```
GC heap initialization failed with error 0x8007000E
Failed to create CoreCLR, HRESULT: 0x8007000E
```

Un mismo prototipo puede divergir por dispositivo y estado instalado

Redmi Note 8

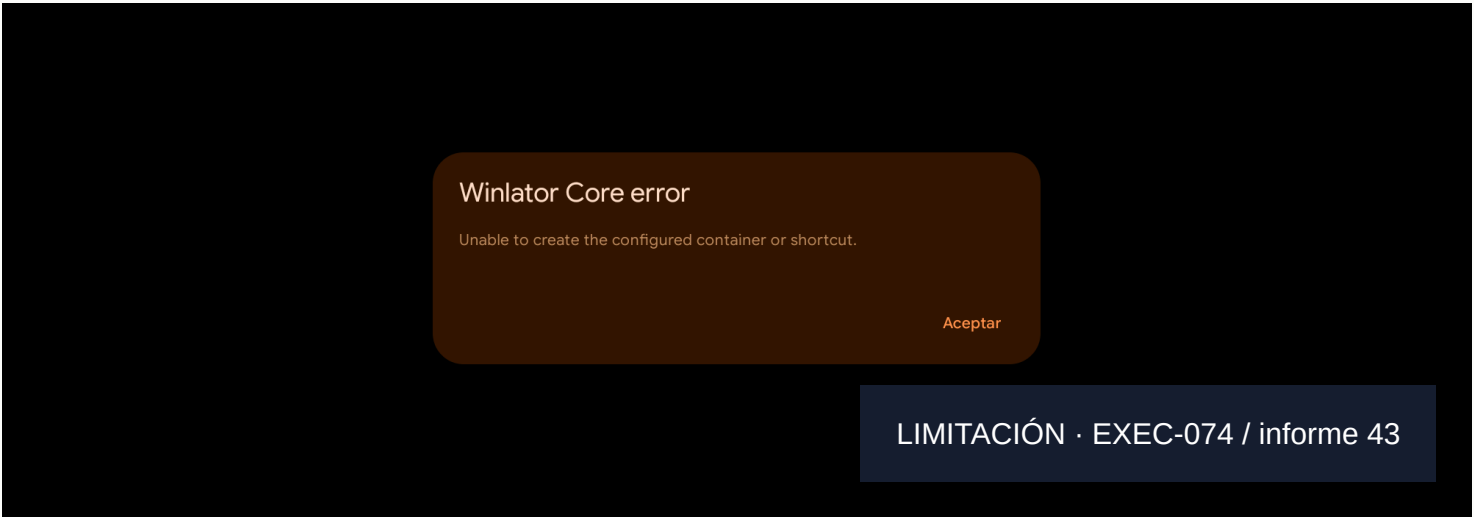
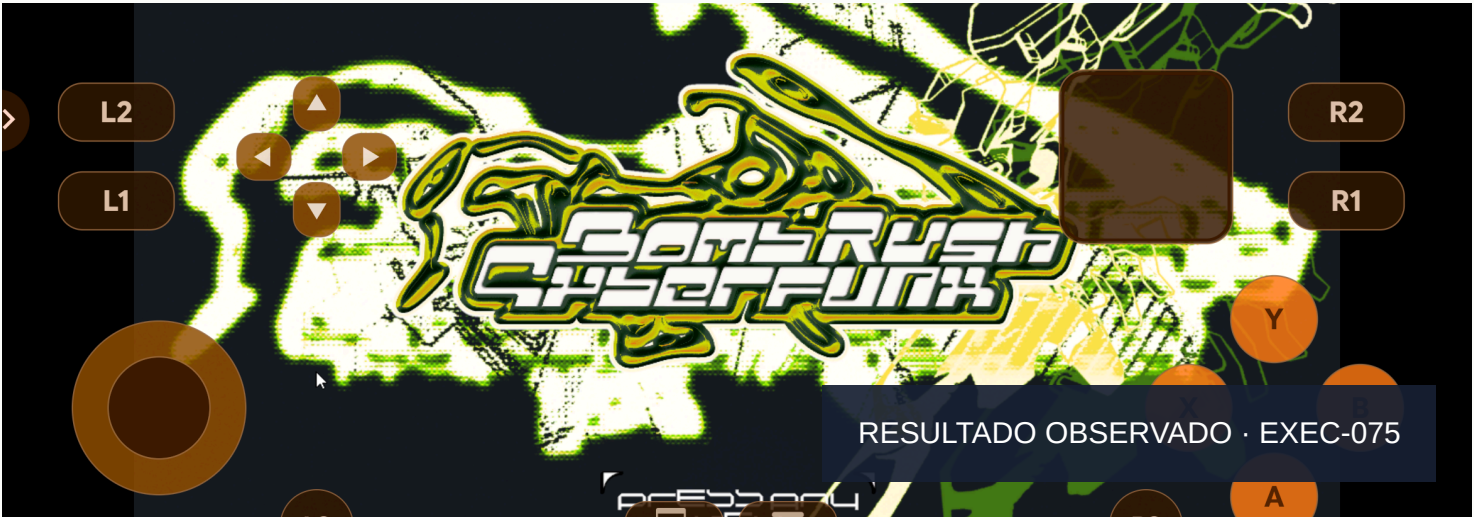
Pantalla de título, gamepad visible y procesos observables durante 120 s.

Lenovo TB-J606F

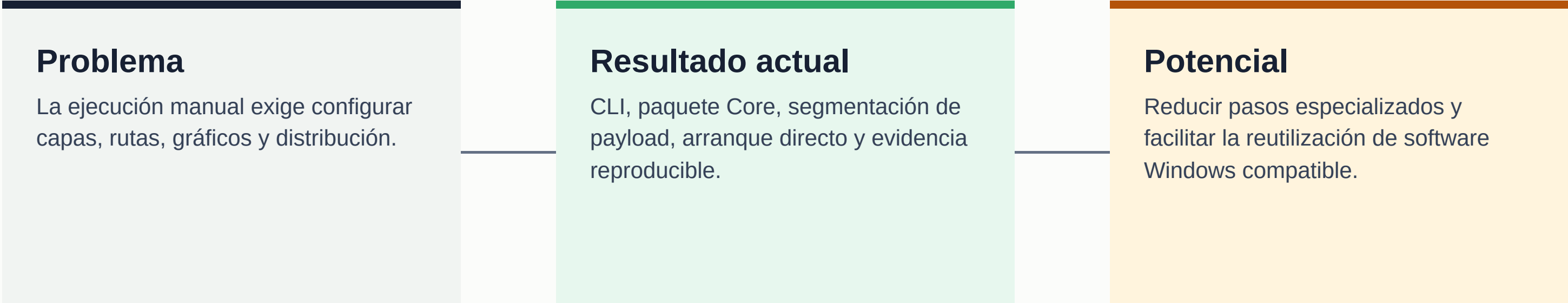
Error al recuperar win2apk_payload_002 desde el proveedor de pruebas locales.

Decisión

Demo principal en Redmi; Lenovo queda como evidencia de una limitación reproducida.



El aporte medido es técnico; el impacto de adopción aún es potencial



No afirmamos todavía: compatibilidad universal, ahorro económico cuantificado, reducción ambiental ni adopción por usuarios finales.

¿QUÉ DEMOSTRAMOS HASTA AHORA?

Win2APK ya es un sistema experimental que construye, distribuye, ejecuta y aprende de sus límites.

Existe: CLI, runtime configurado, artefactos y protocolo de medición.

Validamos: aplicaciones de prueba, payloads grandes y ejecución en cuatro dispositivos.

Permanece: compatibilidad variable, publicación moderna y validación con más aplicaciones y usuarios.



FUERA DE LOS 7 MINUTOS

Backup / Preguntas

Arquitectura, criterios, dispositivos, métricas, limitaciones,
licencias y fuentes.

Use ← y → para navegar B1

Qué hace cada capa y dónde termina Win2APK

Canalización propia

- CLI Rust y esquema JSON v2.
- Inventario, hashes, segmentación y staging.
- Identidad de paquete, icono y firma.
- Configuración Core, acceso directo y arranque.
- Entrega, movimiento y limpieza de una copia final.
- Perfiles gráficos y protocolo de medición.

Componentes integrados

- **Winlator:** base Android y entorno de ejecución.
- **Wine:** API Windows sobre servicios POSIX.
- **Box64:** traducción de código x86-64 en hosts ARM64.
- **DXVK / WineD3D:** traducción gráfica.
- **Mesa / Turnip / Vortek:** acceso a GPU según dispositivo.
- **Android App Bundle / PAD:** distribución del payload.

Qué se considera evidencia y qué no

Criterios observables

- Artefacto y SHA-256 identificados.
- Instalación o modo preinstalado explícito.
- Primer inicio y segundo inicio separados.
- Proceso de app y proceso objetivo visibles.
- Captura canónica T+60 clasificada.
- CPU, memoria, temperatura y almacenamiento cuando están disponibles.
- Logs y video conservados.

Reglas de interpretación

- Una pantalla negra no prueba por sí sola un cierre.
- Sin fila de instalación no significa fallo si el modo es preinstalado.
- FPS de carga o menú no equivale a jugabilidad.
- Un dispositivo no representa compatibilidad universal.
- Una hipótesis causal requiere una prueba que la aísle.
- Los CSV históricos no se reescriben para “mejorar” un resultado.

Cuatro perfiles, no una muestra estadística de Android

EQUIPO	SOC / GPU	FACTOR	USO EN EL PROYECTO	LECTURA CORRECTA
Google Pixel 9a	Tensor / Mali-G715	Teléfono	Memoria, drivers y BCn	Familia Mali moderna
Redmi Note 8	Snapdragon 665 / Adreno 610	Teléfono	Arranque, UI y gamepad	Demo actual
Xiaomi Mi A3	Snapdragon 665 / Adreno 610	Teléfono	PAD y payload grande	Entrega verificada
Lenovo TB-J606F	Snapdragon 662 / Adreno 610	Tablet	Core, layout y fallos	Factor tablet

Propósito de la muestra: encontrar variación de GPU, sistema y factor de forma. No permite estimar una tasa poblacional de compatibilidad.

El límite de 4.45 GiB cambió la estrategia de empaquetado

Fallo inicial

Un APK único no pudo comprimir un asset de 4,782,573,186 bytes y produjo `Required array size too large`.

La evidencia descartó el enfoque monolítico para payloads de ese tamaño.

Resultado posterior

La CLI planificó Cuphead en asset packs y produjo:

- AAB: 5,494,326,036 bytes.
- APKs: 6,424,836,814 bytes.
- Entrega observada: 815 archivos, 5.847 GB.
- Una copia final después de la limpieza.

Lo que todavía impide hablar de producto terminado

Compatibilidad y gráficos

- Resultados dependientes de aplicación, GPU, driver y versión.
- Pixel 9a presentó presión de memoria, pantallas negras y requisitos ADB en variantes.
- Mi A3 mantuvo pantalla negra en una prueba reciente del gamepad.
- Lenovo reprodujo un Unity Crash Handler y, después, un pack local-testing ausente.

Distribución y validación

- `targetSdk 36` bloqueó experimentalmente la ejecución de Box64 desde datos de app; el prototipo actual usa 28.
- Ese objetivo no satisface los requisitos modernos de publicación en Google Play.
- No hay prueba de usuarios ni impacto económico medido.
- Faltan más aplicaciones, reinstalaciones frías y criterios de éxito agregados.

El fallo de Lenovo está localizado, no resuelto

```
E/Win2APKAssetPack:  
java.util.concurrent.ExecutionException:  
com.google.android.play.core.common.LocalTestingException:  
No APKs available for pack  
'win2apk_payload_002'.
```

Resultado visual:
"Unable to create the configured
container or shortcut."

Método:
preinstalled-no-reinstall
Duración: 120 s
Captura: T+60

Qué sabemos

- Play Core no encontró el APK del pack solicitado.
- El error ocurre antes de preparar el contenedor.
- El estado instalado de Lenovo no sirve para la demo.

Qué no sabemos aún

- Si la causa es residuo, instalación incompleta o variante.
- Si una instalación fría controlada lo elimina.

Compatibilidad, traducción y distribución no son sinónimos

Distinciones

- **Wine:** traduce llamadas de API Windows a servicios POSIX; no es una máquina virtual.
- **Box64:** ejecuta código x86-64 de espacio de usuario sobre un host ARM64.
- **DXVK:** traduce Direct3D 8-11 a Vulkan para Wine.
- **Win2APK:** empaqueta y configura esas capacidades; no recompila la aplicación.

Licencias y responsabilidad

- El prototipo integra proyectos de código abierto con licencias propias.
- La distribución debe conservar avisos y revisar compatibilidad de licencias por componente.
- Win2APK no concede derechos sobre aplicaciones Windows empaquetadas.
- El usuario debe tener autorización para redistribuir el software y sus datos.

Fuentes primarias y evidencia local

1. Android Developers. (2026). *About Android App Bundles*.
<https://developer.android.com/guide/app-bundle>

2. Android Developers. (2026). *Play Asset Delivery*.
<https://developer.android.com/guide/playcore/asset-delivery>

3. Android Developers. (2026). *Test asset delivery*.
<https://developer.android.com/guide/playcore/asset-delivery/test>

4. Box64 contributors. (2026). *Box64* [Software]. GitHub. <https://github.com/ptitSeb/box64>

5. Rebohle, P., et al. (2026). *DXVK* [Software]. GitHub. <https://github.com/doitsujin/dxvk>

6. The Wine Project. (2026). *About Wine*. <https://www.winehq.org/about>

7. Winlator contributors. (2026). *Winlator* [Software]. GitHub. <https://github.com/brunodev85/winlator>

8. Win2APK. (2026). *Ejecuciones 001-075, especificaciones, limitaciones y código fuente* [Repositorio de investigación].

9. Win2APK. (2026). *Revisión PRISMA: 98 PDF, 92 estudios únicos y 48 estudios incluidos* [Informe interno].

Trazabilidad completa: consulte docs/fuentes.md para URL, uso en diapositiva, archivos locales y alcance de cada afirmación.